

Tiny Transformer for Next-Token Prediction on Shakespeare Text

In this assignment, I create a tiny transformer for next-token prediction using the Shakespeare Tiny Corpus. The goal is to build a lightweight implementation of core Transformer components and train the model to predict the next subword token in a sequence of Shakespeare text. The objective is to understand how tokenization, positional encodings, self-attention, and feed-forward layers work together to perform language modeling on a small dataset.

First, I use Byte-Pair Encoding to convert the raw Shakespeare text into subword tokens. Byte-Pair Encoding is effective because it captures meaningful text units (e.g., prefixes, suffixes, common word fragments) while avoiding the overly large vocabulary required by full word-level tokenization.

To perform Byte-Pair Encoding, I first pre-tokenize the corpus into words using simple whitespace splitting. I compute the frequency of each word in the corpus and create a base alphabet containing every unique character appearing in the corpus. I then split each word into its individual characters in order to be able to start training.

At each step in the training, I compute the frequency of all adjacent symbol pairs (characters or subwords). The most frequent pair is merged into a new subword and added to the vocabulary. This process is repeated until the vocabulary size reaches the target size of about 300 tokens. The final vocabulary list is composed of the special token (<|endoftext|>), the

Merges: {('t', 'h'): 'th', ('o', 'u'): 'ou', ('e', 'r'): 'er', ('i', 'n'): 'in', ('a', 'n'): 'an', Final Vocabulary: ['< endoftext >', '!', '\$', '&', '"', ',', '-', '.', '3', ':', ';', '?', 'A', 'B', 'C', 'D'
--

initial vocabulary, and the result of all the merges. I then use this vocabulary to split the entire Shakespeare corpus into subwords.

Since computers cannot understand text directly, the subwords are converted into unique integer IDs based on the vocabulary. This produces a numeric representation of the text

Vocabulary size: 298 Total tokens: 495,887 Sample tokens: [("'", 0), ('\$ ', 1), ('&', 2), ('"', 3), (''d"', 4), (''ll"', 5), (''s"', 6), (''t"', 7), (' ', 8), ('-', 9)]

that preserves subword-level structure. After the tokenization, the vocabulary size is 298 and there are 495,887 total tokens.

To train the Transformer, I transform the linear list of token IDs into fixed-length sequences of 50 tokens. Each input sequence is paired with a target sequence that is shifted by one position, forming input–target pairs for next-token prediction. See below for an example of

Example Input IDs: [113, 193, 136, 230, 117, 131, 270, 270, 202, 194] Example Target IDs: [193, 136, 230, 117, 131, 270, 270, 202, 194, 106]

an input-target pair. Note that this example only shows the first 10 tokens in the sequence. I then split the data into an 80% training set and 20% validation set.

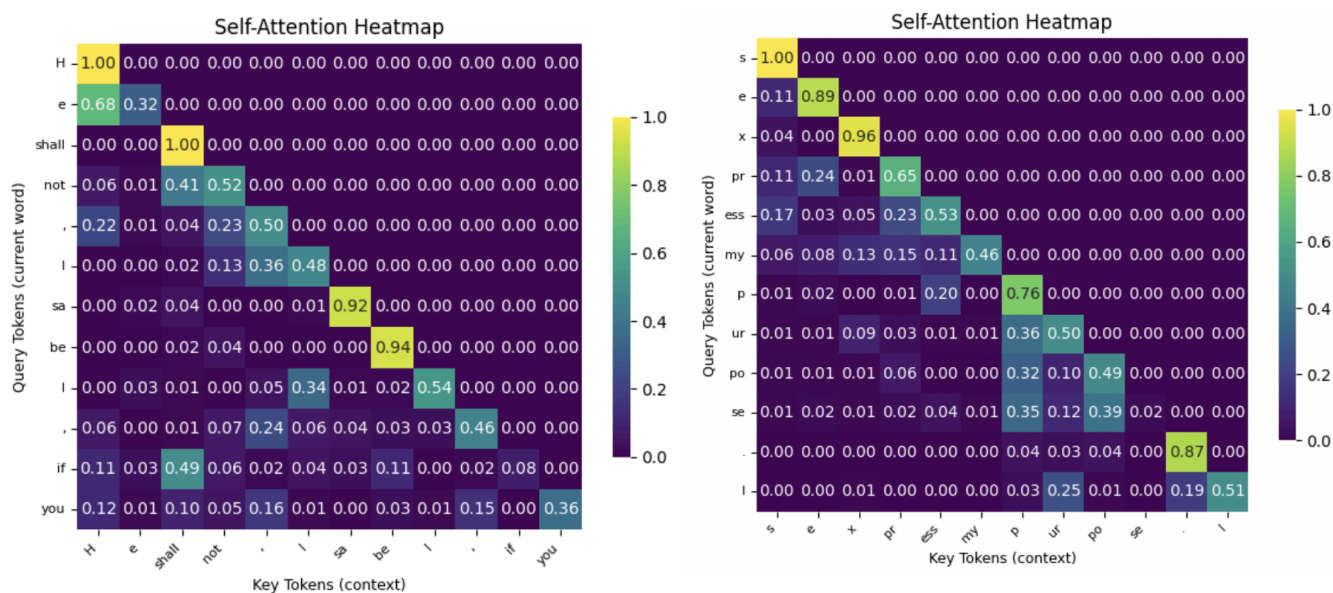
Transformers cannot operate directly on integer token IDs, so I convert each unique token ID into a vector of numeric values called an embedding. Ideally, these vectors capture semantic meaning, where words or subwords that are similar in meaning have embeddings that are closer together in the vector space.

To create the embeddings, I use `nn.Embeddings` to map each integer token to a 64-dimensional vector. The Transformer can now operate on these embeddings. However, because embeddings alone contain no information about word order, I add learned positional encodings. Without positional encodings, the model would treat sequences as bags of tokens, and attention would not distinguish between different positions. Including position information allows the Transformer to reason about sequential structure.

In addition to the token embeddings and positional encodings, my Transformer includes two Transformer blocks, each containing a self-attention layer with a causal mask, RMSNorm applied before both the attention and feed-forward sublayers, a feed-forward network with a hidden dimension of 256, and residual connections after each sublayer.

When humans read sentences, we do not process each word equally. Rather, we focus on the words that carry the most meaning. The self-attention mechanism is designed to mimic this behavior. Each token computes attention weights that determine how much it should “focus” on every other token in the same sequence. By comparing each token’s embedding to all others, the model computes similarity scores, which are used to form a weighted average of the surrounding context. This produces context-aware representations that help the model understand relationships across the entire sequence before predicting the next word.

During next-token prediction, however, we do not want the model to see future tokens. Thus, I apply a causal mask that guarantees autoregressive generation, where the model learns to predict the next token solely from the past context. I then use a residual connection to preserve the original information while adding the newly learned contextual information from the self-attention output.



The heatmaps above show the attention patterns learned by the model. In the heatmap on the left, we have the sequence, “He shall not, Isabel, if you...”. In the first row, “H” only attends to itself, since causal masking prevents it from attending to any tokens to its right. This masking ensures that each token can only see the tokens that came before it in the sequence.

In row 2, token “e” attends most strongly to “H” (attention weight = 0.68) and moderately to itself, suggesting the model is learning to form the complete word “He.” Further down at row 11, the token “if” attends strongly to “shall”.

Looking at the heatmap on the right of the sequence “...s express my purpose. I...”, I see similar behavior: most tokens attend most strongly to themselves, but some also attend to neighboring subwords. In row 9, token “ur” attends strongly to both itself and the subword before it, “p”. Token “po” in row 10 attends strongly to “p”, “ur”, and itself. Lastly, token “se” in row 11 attends strongly to “p”, “ur”, and “po”. This pattern makes sense, as these subwords together form the word “purpose.”

Overall, the self-attention heatmaps show that most tokens attend primarily to themselves, as seen in the bright diagonals, while also attending to nearby subwords when constructing full words (e.g., “p-ur-po-se”). Occasionally, there are also long-range dependencies (e.g., “if” attending to “shall”), showing that the model captures both local structure and broader semantic relationships within the text.

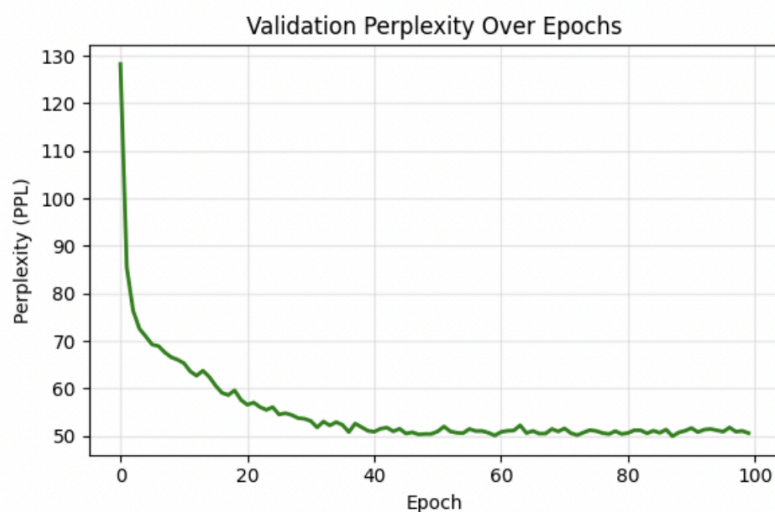
After the self-attention step, each token embedding has context information, but the representation is still linear. In order for the model to capture more complex patterns, I use a feed-forward network. The feed-forward network applies two linear transformations with a ReLU activation to each token embedding, allowing the model to learn complex, non-linear patterns. RMSNorm normalizes the input to maintain numerical stability, and the residual connection adds the original input back to the output to preserve information and stabilize training.

Finally, I train the model using a hidden size of 64, 2 transformer blocks, a feed-forward size of 256, and a max sequence length and batch size of 128. To determine the epoch giving the best model, I will use both training loss and perplexity. Perplexity is the measurement of how well a probability model predicts a sample. It captures how uncertain a model is in predicting the next word in a sequence. The smaller the perplexity, the better the model. Using training loss and perplexity to determine the best epoch, the model performs best at epoch = 88, as the

training loss and perplexity is the lowest. At epoch 88, the training loss is 3.09, validation loss of 3.91, and perplexity 49.94. This can be seen in the training vs. validation loss curves. Initially, the losses are high, but as the model trains for more epochs, both losses gradually decrease. Eventually, the two curves begin to flatten out, which indicates a good stopping point since we do not want one of the curves to start rising, as that would indicate overfitting. A good stopping



point in this case is around epoch 88, where the validation loss reaches its lowest value before leveling off.



A similar pattern is seen in the perplexity curve over epochs. Perplexity starts off large and eventually levels off, with the lowest perplexity score being at epoch 88.

Transformers typically require more computation and data than simpler models, so I tracked runtime and memory usage during training. The total training time was 1113.56 seconds (18.6 minutes) with the maximum memory used being 1.03 GB. The main runtime bottleneck was the self-attention computation. This is known to be costly since it requires building an $L \times L$ attention matrix that has quadratic complexity in sequence length. As for memory, most usage came from storing activations needed for backpropagation. This is also a common bottleneck in Transformer training because activation sizes scale with batch size, sequence length, hidden size, and number of layers. This produces millions of values held in the memory at once. Even though my model is small ($d_{\text{model}}=64$, $n_{\text{layers}}=2$), these two components still dominated both runtime and memory.

In order to assess how the model performs, I generate two samples. The model created sentences similar to how Shakespeare wrote, including actual names of people in his text such

Sample 1:
shall be con te m b s ; And th us po the king And that he sa id my self ma st er with him . Q U E E N M A R G A R E T : A ll : ' K is w all . M O P O M
Sample 2:
My L ord of N or f o l k . I f shall have him be thy self . K I N G H E N R Y V I : H ow c ould I we p l come her . W A R W I C K : O ut u s, e shall

as Queen Margaret and King Henry VI. However, there are some instances of words created with some errors; for example, the model wrote “welpcome” instead of “welcome”. Although errors like this are expected given the small model size, only two Transformer layers, and relatively short training budget. Nevertheless, the samples demonstrate that the model has learned stylistic and structural patterns from the corpus.

Overall, the tiny Transformer successfully learned meaningful patterns from the Shakespeare corpus. The model captured both local subword structure and some long-range dependencies, as shown by the attention visualizations. Training was stable, and the model achieved reasonable perplexity for a model of this size.